



SERVIÇO PÚBLICO FEDERAL · MINISTÉRIO DA EDUCAÇÃO

UNIVERSIDADE FEDERAL DE VIÇOSA · UFV

CAMPUS FLORESTAL

Trabalho prático 1: Projeto e análise de algoritmos

Jornada Espacial

Pâmela Lúcia Lara Diniz [5898]

Beatriz Queiroz Cruz Silva [5890]

Maria Eduarda Oliveira Garcia Andrade [5365]

Eduarda Anaely Suar Borges [5911]

Florestal - MG

2025

Sumário

1. Introdução	3
2. Organização	3
3. Desenvolvimento	4
3.1 Entrada.c	5
3.2 Pecas.c	7
3.3 Rota.c	7
3.4 ControleNave.c	9
3.5 Main.c	13
3.6 Modo de análise	15
4. Tarefas Extras	16
4.1 Interface Extra	16
4.2 Geração de Mapas	17
4.3 Desafios Extras	18
5. Compilação e Execução	19
6. Resultados	19
7. Conclusão	25
8. Referências	26

1. Introdução

O assunto abordado nesta documentação é o desenvolvimento de um programa na linguagem C para ajudar a tripulação a encontrar uma rota para chegar ao planeta das festividades, utilizando Backtracking. Atendendo às especificações do trabalho prático, foram implementadas funções responsáveis por ler o arquivo de entrada, alocar dinamicamente o mapa da rodovia estelar e identificar as posições iniciais e finais. A partir desses dados, o programa utiliza uma função recursiva que aplica o paradigma de Backtracking para explorar todos os caminhos possíveis entre o ponto inicial e o planeta das festividades, considerando as restrições de durabilidade, perda por movimento e ganho ao coletar peças. Além disso, foram implementadas as tarefas extras propostas na documentação do trabalho prático, como a interface adicional, a opção de geração de mapas e dois desafios para a nave: uma chuva de meteoros e ladrões extraterrestres. Por fim, este relatório contém uma apresentação detalhada do programa criado e dos resultados obtidos.

2. Organização

Na Figura 1 é possível visualizar a organização do projeto. Na pasta **include/** encontra-se os arquivos *header* que contém a definição de todos os TADs e funcionalidades utilizadas, enquanto o diretório **src/** armazena a implementação dessas mesmas estruturas e funções. Já a pasta **testes/** é onde ficam armazenados os arquivos de teste do programa, que devem seguir o formato **.txt**.

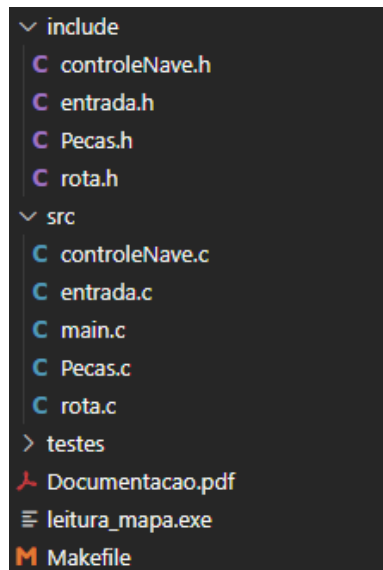


Figura 1 - Repositório do projeto.

Quanto aos arquivos localizados diretamente no diretório raiz, existe um **Makefile** com os comandos necessários para compilar e executar o programa, um executável **leitura_mapa.exe** que é gerado após a compilação e, por fim, a documentação do projeto. Na figura 2 está a estrutura de dados utilizada para compor o projeto:

```
typedef struct {
    int D; //durabilidade inicial
    int Dperda; //quanto perde por movimento
    int Againho; //quanto ganha ao pegar uma peça
    int altura; //linhas
    int largura; //colunas
    char **mapa; //matriz dinâmica com o mapa
    int linhaInicial; //posição do 'X'
    int colunaInicial; //posição do 'X'
    int qtdPecas; //guarda quantas pecas ja foram pegadas
    int linhaFinal;
    int colunaFinal;
} Mapa;
```

Figura 2 - Struct Mapa.

3. Desenvolvimento

Serão percorridas nessa seção as implementações e pontos cruciais do desenvolvimento do projeto.

3.1 Entrada.c

Aqui estão presentes todas as funções relacionadas à leitura do arquivo.

- lerArquivo(): Aqui é criada dinamicamente uma instância da estrutura Mapa, que armazenará todas as informações do jogo (parâmetros de durabilidade, mapa, posições, etc.), além da alocação dinâmica da matriz, que representará o mapa do jogo. Também acontece a identificação da posição inicial 'X' e final 'F'.

```
Mapa* m = (Mapa*) malloc(sizeof(Mapa));
if (!m) {
    printf("Erro de alocação de memória para Mapa.\n");
    fclose(arquivo);
    return NULL;
}

//lê os valores iniciais
fscanf(arquivo, "%d %d %d", &m->D, &m->Dperda, &m->Againho);
fscanf(arquivo, "%d %d", &m->altura, &m->largura);

//aloca a matriz do mapa
m->mapa = (char**) malloc(m->altura * sizeof(char*));
for (int i = 0; i < m->altura; i++) {
    m->mapa[i] = (char*) malloc((m->largura + 1) * sizeof(char)); // +1 pro '\0'
}

for (int i = 0; i < m->altura; i++) {
    fscanf(arquivo, "%s", m->mapa[i]);
}

fclose(arquivo);

//localiza a posição inicial 'X' e final 'Y'
m->linhaInicial = -1;
m->colunaInicial = -1;

m->linhaFinal=-1;
m->colunaFinal=-1;

int fim=0;

for (int i = 0; i < m->altura && !fim; i++) {
    for (int j = 0; j < m->largura; j++) {
        if (m->mapa[i][j] == 'X') {
            m->linhaInicial = i;
            m->colunaInicial = j;
        }
        else if (m->mapa[i][j] == 'F') {
            m->linhaFinal = i;
            m->colunaFinal = j;
        }
    }
}
```

Figura 3 - Implementação da leitura do arquivo e alocação da matriz.

- imprimirMapa(): Responsável por exibir o mapa no terminal com cores e informações completas.

```

for (int i = 0; i < m->altura; i++) {
    for (int j = 0; j < m->largura; j++) {
        char c = m->mapa[i][j];
        switch (c) {
            case 'X': printf(RED "X" RESET); break;
            case 'F': printf(GREEN "F" RESET); break;
            case 'P': printf(YELLOW "P" RESET); break;
            case '-': printf(CYAN "-" RESET); break;
            case '|': printf(CYAN "|" RESET); break;
            case '+': printf(MAGENTA "+" RESET); break;
            case '.': printf(GRAY "." RESET); break;
            default: printf(WHITE "%c" RESET, c); break;
        }
    }
    printf("\n");
}

```

Figura 4 - Implementação da impressão do mapa.

- gerarCaractereCaminho(): De acordo com o nível de dificuldade (1 = fácil, 2 = médio, 3 = difícil), a função altera as probabilidades de surgimento de obstáculos e caminhos. Isso faz com que mapas mais difíceis tenham menos caminhos livres e mais obstáculos, o que influencia diretamente na quantidade de backtracking necessária.

```

char gerarCaractereCaminho(int dificuldade) {
    int r = rand() % 100;
    switch (dificuldade) {
        case 1: // fácil
            if (r < 60) return '-';
            else if (r < 80) return '|';
            else if (r < 90) return '+';
            else return '.';
        case 2: // médio
            if (r < 40) return '-';
            else if (r < 55) return '|';
            else if (r < 65) return '+';
            else return '.';
        case 3: // difícil
            if (r < 25) return '-';
            else if (r < 35) return '|';
            else if (r < 40) return '+';
            else return '.';
        default:
            return '.';
    }
}

```

Figura 5 - Implementação da função gerarCaractereCaminho.

3.2 Peças.c

No arquivo de Peças estão implementadas quatro funções relacionadas aos eventos que ocorrem quando uma peça é encontrada, quando a nave se movimenta pelo mapa, quando ela é atingida por uma chuva de meteoros ou quando é saqueada. Todas essas funções são chamadas no arquivo ControleNave.c, especificamente dentro da função movimentarNave.

A função ConcertarNave é chamada quando uma peça é recolhida durante a trajetória. Ela contabiliza quantas peças já foram obtidas e adiciona o valor da peça (A) à durabilidade da nave.

A função AndarNoMapa é uma das mais importantes para a lógica do código, pois é responsável por subtrair da durabilidade o gasto (D') referente ao deslocamento por um setor da rodovia. Inicialmente, ela verifica se as quatro peças já foram recolhidas; caso positivo, apenas retorna 1, indicando que a nave andou com sucesso, pois já está completamente consertada e, portanto, sua durabilidade não diminui mais. Caso contrário, verifica se a nave possui durabilidade suficiente para continuar. Se sim, subtrai o valor gasto ao andar da durabilidade e retorna 1; se não, retorna 0, o que significa que a nave não tem mais durabilidade suficiente para se mover.

As funções UmMeteoroAcertou e RouboExtraterrestre estão relacionadas aos pontos extras e serão discutidas na Seção 4 – Tarefas Extras.

3.3 Rota.c

O arquivo rota.c conta com a implementação de uma lista encadeada que armazena posições, como definido na estrutura da figura 6.

```
typedef struct Posicao{
    int linha;
    int coluna;
    int durabilidade;
    int pRestantes;
    struct Posicao* prox;
}Posicao;

typedef struct RotaFinal{
    Posicao* primeiro;
    Posicao* ultimo;
}RotaFinal;
```

Figura 6 - Estruturas da lista encadeada RotaFinal.

Essa lista é responsável por armazenar o percurso correto da nave, caso exista, e conta com algumas funcionalidades de inicialização, inserção, impressão e liberação de memória.

```
RotaFinal* inicializaRotaFinal();
Posicao* criaPosicao(int linha,int coluna, int durabilidade, int pRestantes);
void registraPosicao(RotaFinal* rota, int linha, int coluna, int durabilidade, int pRestantes);
void imprimeRota(RotaFinal* rota);
void liberaRota(RotaFinal* rota);
```

Figura 7 - Funções relacionadas à lista encadeada.

Os principais fatores que levaram à utilização de uma lista encadeada foram a necessidade de guardar múltiplas informações de cada movimento da nave, como posição, durabilidade, peças,etc., portanto, a implementação de uma struct se viu necessária, juntamente com uma estrutura para armazená-las. Além disso, essas informações eram obtidas durante a volta das chamadas recursivas quando um caminho possível era encontrado, isso fez com que as informações fossem obtidas da posição final até a posição inicial, ou seja, na ordem inversa. Logo, para armazenar uma quantidade variável de posições e de forma que cada uma pudesse ser inserida no início da lista para manter a ordem correta do percurso sem a

necessidade de ordenação, a lista encadeada se mostrou como uma implementação ideal.

3.4 ControleNave.c

Esse arquivo implementa toda a lógica principal de tomada de decisões e movimentação da nave, tendo como principal característica a utilização da técnica de backtracking.

A função **encontraCaminho** é uma implementação auxiliar que é responsável por alocar memória e inicializar estruturas necessárias para que seja feita a busca pelo percurso. Uma vez que isso é feito, chama **movimentarNave** e, de acordo com o resultado obtido na busca, informa ao usuário e, por fim, faz a liberação da memória utilizada no procedimento.

A função **defineDirecoes** auxilia na identificação de quais caminhos podem ser tomados a partir da sua posição atual. Por exemplo, caso se encontre em um '-' essa função irá limitar as verificações horizontalmente para a esquerda e direita, no caso de '|', as possibilidades de caminho serão para cima e para baixo e por fim, uma encruzilhada ou objeto especial indica verificação nas quatro direções.

```
void defineDirecoes(char c, char *vetor, int* possibilidades) {  
  
    char encruzilhada[] = {'e', 'd', 'c', 'b', '\0'};  
    char horizontal[]   = {'e', 'd', '\0'};  
    char vertical[]     = {'c', 'b', '\0'};  
  
    switch (c) {  
        case '-':  
            strcpy(vetor, horizontal);  
            (*possibilidades)=2;  
            break;  
        case '|':  
            strcpy(vetor, vertical);  
            (*possibilidades)=2;  
            break;  
        default:  
            strcpy(vetor, encruzilhada);  
            (*possibilidades)=4;  
            break;  
    }  
}
```

Figura 8 - implementação da função defineDirecoes.

A função **verificaMovimento** também auxilia na identificação do percurso. A partir de uma direção recebida como parâmetro, ela faz o cálculo da nova posição ocupada pela nave. Por exemplo, desloca uma posição para esquerda, direita, cima ou baixo. Então, é verificado se aquele posicionamento é inválido, ou seja, se caiu fora das dimensões da matriz, se é uma parede ou se aquela posição já foi visitada nessa mesma chamada recursiva(pois isso pode causar um loop), a função retorna que o movimento não deve ser feito.

```
int verificaMovimento(char direcao, Mapa* mapa, int x, int y, int* novoX, int* novoY, int** visitado){

    *novoX=x;
    *novoY=y;

    switch (direcao){ //calcula a nova posicao que vai ocupar na matriz de acordo com a direcao
        case 'e':
            (*novoY)--;
            break;
        case 'd':
            (*novoY)++;
            break;
        case 'c':
            (*novoX)--;
            break;
        case 'b':
            (*novoX)++;
            break;
    }

    if (*novoX < 0 || *novoX >= mapa->altura || *novoY < 0 || *novoY >= mapa->largura)
        return 0; //caiu fora das dimensões da matriz

    if (visitado[(*novoX)][(*novoY)]){
        return 0; //posição já visitada anteriormente (já foi verificada recursivamente)
    }

    if (mapa->mapa[*novoX][*novoY] == '.')
        return 0; //encontrou uma parede

    return 1;
}
```

Figura 9 - implementação da função verificaMovimento.

A função **movimentarNave** é a parte principal do programa, se trata de uma função recursiva de busca com a utilização da técnica de backtracking. Em relação aos seus parâmetros, ela receberá o mapa, a matriz 'visitado' para identificar posições já verificadas em uma chamada atual, o que é importante para evitar loops, a matriz 'solucao' que irá identificar posições que fazem parte do percurso

final e irá auxiliar em uma posterior representação visual dele, as coordenadas atuais 'x' e 'y' e uma lista encadeada 'rota' para guardar o percurso final junto com suas informações, independente da quantidade de posições percorridas.

A primeira etapa executada na função é a verificação da ocorrência de algum evento na posição atual que pode alterar o estado da busca, como a coleta de peças ou encontro com algum obstáculo, a lógica por trás disso já foi detalhada na seção Peças.c.

```
char evento = mapa->mapa[x][y];

switch (evento) {
case 'P':
    ConcertarNave(mapa);
    pecaColetada = true;
    break;
case '#':
    UmMeteoroAcertou(mapa);
    danoMeteoro = true;
    break;
case '&':
    pecasRoubadas = RouboExtraterrestre(mapa);
    break;
}
```

Figura 10 - Verificação de eventos.

Posteriormente, é verificada a condição de parada, o caso base da recursão, que é se a posição de destino foi ou não alcançada. Em caso positivo, marca aquela posição na matriz de solução, faz o registro dela na rota final e retorna sucesso, fazendo a recursão subir para a chamada anterior.

```
if (x == mapa->linhaFinal && y == mapa->colunaFinal) {
    solucao[x][y] = 1;
    registraPosicao(rota,x,y,mapa->D,4-mapa->qtdPecas);
    if (g_modulo_analise == 1) g_profundidade_atual--; // Diminui um nível ao sair
    return 1;
}
```

Figura 11 - Caso base da recursão.

E então, existe o trecho que realiza a exploração recursiva, destacado na figura 12. Ele verifica se a nave está em condições de se movimentar, em caso positivo marca a posição como visitada e como parte da solução atual, então são definidas as direções em que a nave pode se movimentar e para cada uma dessas posições é analisado se o movimento é válido. Em caso afirmativo, a nave se move e a função é chamada recursivamente para essa nova posição até que um caminho retorne sucesso e isso se propague para as chamadas anteriores.

```
if (mapa->D <= 0 && mapa->qtdPecas < 4) {
    // Pula para o bloco de reversão no final da função
} else {
    visitado[x][y] = 1;
    solucao[x][y] = 1;

    char direcoes[5];
    int possibilidades=0;
    int novoX, novoY;

    defineDirecoes(mapa->mapa[x][y], direcoes, &possibilidades);

    for (int i = 0; i < possibilidades; i++) {

        if (verificaMovimento(direcoes[i], mapa, x, y, &novoX, &novoY, visitado)) {
            AndarnoMapa(mapa);

            if (movimentarNave(mapa, visitado, solucao, novoX, novoY, rota)) {
                if (g_modos_analise == 1) g_profundidade_atual--; // Diminui um nível ao sair
                registraPosicao(rota, x, y, dAtual, pRestanteAtual);
                return 1;
            }

            if (mapa->qtdPecas < 4) {
                mapa->D += mapa->Dperda;
            }
        }
    }
}
```

Figura 12 - Exploração recursiva dos caminhos.

Por fim, caso se chegue a um caminho inválido dentro de alguma chamada, é necessário retroceder a rota para que seja possível verificar outras direções, isso é justamente o conceito de **backtracking**, que irá desfazer a ocorrência dos eventos, restaurando os atributos da nave, desmarcando a posição como parte da solução e também dos visitados (pois como o estado da nave varia para cada rota realizada, é possível chegar nesse ponto a partir de um percurso diferente em que a nave se encontre em outro estado e que pode levar a uma solução, por isso não é possível

‘cortar’ a verificação dessa posição para outros percursos) e então irá retornar que aquela rota é inválida, retornando para o nível anterior da recursão para verificar outras rotas possíveis. Quando não for mais possível realizar verificações e o caminho ainda não foi encontrado, significa que não existe um percurso válido.

```
}

if (pecaColetada) {
    mapa->qtdPecas--;
    mapa->D -= mapa->Againho;
    mapa->mapa[x][y] = 'P';
}
if (danoMeteoro) {
    mapa->D += mapa->Dperda;
}
if (pecasRoubadas > 0) {
    mapa->qtdPecas += pecasRoubadas;
}

if (g_modos_exibicao == 1) {
    printf("...backtracking em (%d,%d)\n", x, y);
}

solucao[x][y] = 0;
visitado[x][y] = 0;

if (g_modos_analise == 1) g_profundidade_atual--; // Diminui um nível ao sair

return 0;
}
```

Figura 13 - Trecho que desfaz as alterações para retroceder.

Existe também a função **exibeCaminho**, que é responsável por imprimir o mapa, ilustrando de vermelho as posições que fazem parte da rota encontrada, a matriz `solucao` foi utilizada para auxiliar essa identificação.

3.5 Main.c

O arquivo *main.c* serve como o ponto de entrada e o controlador central da interação do usuário com o programa. Sua principal função é controlar o fluxo completo do programa, desde a sua inicialização e interação com o usuário até o seu encerramento. Ele também garante que o usuário possa carregar e testar múltiplos mapas de forma contínua, sem a necessidade de reiniciar o executável.

Para atender a este requisito, a estrutura principal foi construída em torno de um laço *do-while*. Este tipo de laço de repetição garante que o bloco de código seja executado pelo menos uma vez, antes de verificar a condição de continuidade. Sua lógica de "executar primeiro, perguntar depois" conectou-se com a nossa necessidade, pois o menu de opções precisava ser exibido ao usuário logo na primeira execução e, em subsequente, após cada operação, continuando o ciclo até o usuário escolher a opção de sair.

Para atender a necessidade de o programa ser capaz de aceitar novos arquivos de entrada depois de cada execução, encerrando apenas quando o usuário desejar, foi feita a separação das ações de **carregar um mapa** e **iniciar a jornada**. Para isso, uma variável ponteiro é mantida fora do laço principal, servindo como uma memória que contém o mapa atualmente carregado. A interface se dá da seguinte forma:

- **Opção 1 - Carregar Mapa:** O usuário fornece o caminho para um arquivo de texto. A função *lerArquivo* é chamada para alocar e preencher a estrutura do mapa. Se um mapa já estiver carregado na memória, ele é primeiro liberado para evitar vazamentos de memória. Uma vez carregado com sucesso, o programa exibe o mapa para confirmação visual do usuário.
- **Opção 2 - Iniciar Jornada:** Esta opção só procede se a variável ponteiro não for nula. Primeiramente, ela apresenta ao usuário as opções de exibição da interface (detalhadas na Seção 4.1). Em seguida, chama a função *encontraCaminho* para iniciar o algoritmo de backtracking. Ao final da execução, ela exibe as mensagens de resultado e recarrega o mapa do arquivo original. Isso "reseta" o estado do mapa, permitindo que o usuário execute a busca novamente com um modo de exibição diferente sem que as alterações da execução anterior (como peças coletadas) interfiram no resultado.
- **Opção 0 - Sair:** Encerra o laço *do-while* e, antes de finalizar o programa, realiza uma última verificação para liberar qualquer mapa que ainda esteja alocado na memória.

Também é oferecida ao usuário a opção 3: Gerar Novo Mapa, especificada na seção 4.2. Esta opção invoca a função *gerarMapaAutomaticamente* e, ao ser selecionada, ela solicita ao usuário diversos parâmetros e, com base nessas informações, um novo mapa aleatório é criado.

3.6 Modo de análise

Conforme especificado nos requisitos do projeto, foi implementado um "Modo de Análise" para permitir a verificação do funcionamento e da eficiência do algoritmo de backtracking. A ativação deste modo é oferecida ao usuário como a primeira interação ao iniciar o programa, permitindo que toda a sessão seja executada com a coleta de métricas ativada ou desativada.

A implementação foi realizada através de variáveis globais para minimizar o acoplamento com a lógica central do algoritmo. As seguintes variáveis foram criadas:

- *g_modo_analise*: Uma flag, ativada no início do programa pela escolha do usuário.
- *g_chamadas_recurativas*: Um contador que armazena o número total de vezes que a função *movimentarNave* é chamada.
- *g_profundidade_maxima*: Um inteiro que rastreia o nível mais profundo que a recursão alcançou durante uma busca.

A coleta de dados é feita através da "instrumentação" da função *movimentarNave* no arquivo *controleNave.c*. No início da função, se o modo análise estiver ativo, *g_chamadas_recurativas* é incrementado. A profundidade atual da recursão também é incrementada e comparada com *g_profundidade_maxima*, atualizando o valor máximo se necessário. Antes de cada return, para manter a contagem de profundidade precisa, a variável que a rastreia é decrementada antes de cada ponto de saída da função. Isso garante que, ao retroceder (backtrack), a contagem de

profundidade reflita corretamente o retorno para um nível anterior na pilha de recursão.

Ao final de cada execução da "Jornada", independentemente do sucesso ou falha, o *main.c* verifica se o modo análise está ativo e, em caso afirmativo, imprime um relatório final com o total de chamadas e o nível máximo de recursão alcançado, como pode ser visto nos resultados.

4. Tarefas Extras

Neste capítulo, iremos nos aprofundar na lógica utilizada para desenvolver os tópicos extras que foram acrescentados no código.

4.1 Interface Extra

Para cumprir a primeira tarefa extra, a interface do programa foi desenvolvida para oferecer ao usuário múltiplas formas de exibir o resultado da busca pelo caminho. Após carregar um mapa e antes de iniciar a jornada, o programa apresenta um submenu com as seguintes opções:

1. **Relatório Detalhado:** Exibe uma visualização completa da jornada, passo a passo, incluindo a posição atual, durabilidade e peças restantes. Também mostra as mensagens de backtracking e os eventos que ocorrem no mapa (chuva de meteoros, roubo de peças).
2. **Relatório com apenas casos de sucessos:** Exibe uma visualização mais 'limpa' mostrando apenas os passos do caminho bem-sucedido e os eventos, omitindo as mensagens de backtracking. Por fim, é exibida também uma representação visual do percurso no mapa.

Para implementar essa funcionalidade foi feito o uso de uma variável global, *g_modos_exibicao*. Nos arquivos *controleNave.c* e *Pecas.c*, a variável é declarada como *extern*, e todas as chamadas *printf* relevantes são envolvidas

por uma estrutura condicional que verifica a escolha do usuário antes de imprimir qualquer informação na tela.

4.2 Geração de Mapas

- gerarMapaAutomaticamente(): Gera automaticamente um novo mapa de teste, pedindo parâmetros ao usuário e salvando o resultado em um arquivo .txt.
 - O usuário escolhe os parâmetros que controlam o comportamento da nave e o tamanho do mapa. Essas informações são usadas para **personalizar o ambiente de teste** e permitir a criação de diferentes cenários para análise de desempenho.
 - O mapa é gerado linha por linha, preenchendo cada célula com um caractere baseado na dificuldade. Essa etapa garante a criação de **testes aleatórios mas controlados**, fundamentais para avaliar o algoritmo.
 - Define aleatoriamente o ponto de partida da nave e o destino final.
 - Esses elementos, '#' e '&' adicionam **eventos aleatórios de perda ou roubo**, aumentando a complexidade do jogo.
 - O mapa final é gravado em um arquivo de texto, no mesmo formato dos arquivos de teste da pasta testes. Isso permite usar o novo mapa diretamente nas execuções automáticas do programa principal.

```
//ADIÇÃO DOS DESAFIOS
int chanceMeteoro = rand() % 2; // 50% de chance
int chanceRoubo = rand() % 2; // 50% de chance
```

Figura 14 - Geração aleatória de desafios.

- O mapa final é gravado em um arquivo de texto, no mesmo formato dos arquivos de teste da pasta testes, também sendo impresso colorido no terminal. Isso permite usar o novo mapa diretamente nas execuções automáticas do programa principal.

```
fprintf(arq, "%d %d %d\n", D, Dperda, Againho);
fprintf(arq, "%d %d\n", altura, largura);
for (int i = 0; i < altura; i++)
    fprintf(arq, "%s\n", mapa[i]);
fclose(arq);

printf("\nArquivo '%s' gerado com sucesso!\n\n", nomeArquivo);
```

Figura 15 - Exibição do novo mapa gerado.

4.3 Desafios Extras

Localizadas no arquivo *Pecas.c*, as funções *UmMeteoroAcertou* e *RouboExtraterrestre*, representadas respectivamente pelos símbolos '#' e '&', são desafios que podem aparecer durante uma rota escolhida.

Quando se viaja pelo espaço, é preciso estar preparado para tudo, mas os engenheiros da nave esqueceram de verificar se os escudos contra meteoros estavam ativados. Por conseguinte, caso um meteoro colida com a nave, ela perderá durabilidade. A função *UmMeteoroAcertou*, após ser chamada, exibe uma mensagem avisando sobre a colisão com o meteoro e, em seguida, verifica se a nave ainda possui durabilidade suficiente. Se possuir, o valor equivalente ao gasto por movimento é subtraído da durabilidade, o que significa que, ao passar pelo símbolo '#', a nave perde o dobro do valor de D' e ao final retorna 1. Caso contrário, retorna 0, indicando que a nave falhou ao seguir por aquela rota.

As peças são itens valiosos naquela região da Via Láctea e, portanto, devem ser bem protegidas contra ladrões. Quando a função *RouboExtraterrestre* é chamada, é sinal de que a nave interestelar foi invadida e que alienígenas estão tentando roubar suas peças. No caso base, é verificado se a nave possui alguma

peça , caso não possua, o roubo não faz sentido e a função retorna sem alterações. Em seguida, é realizada uma probabilidade para determinar se o roubo foi bem-sucedido: se o resultado for 1, a tripulação consegue se defender com sucesso; caso contrário, o roubo é bem-sucedido. Por fim, é feita uma verificação da quantidade de peças disponíveis. Se houver apenas uma, ela é levada e o contador de peças (*qtdPecas*) é zerado. Caso haja mais de uma, uma nova probabilidade é calculada para determinar quantas peças os alienígenas conseguiram roubar antes que os reforços chegassem. Ao final, uma mensagem é exibida informando quantas peças foram roubadas, e o valor correspondente é subtraído do contador de peças.

5. Compilação e Execução

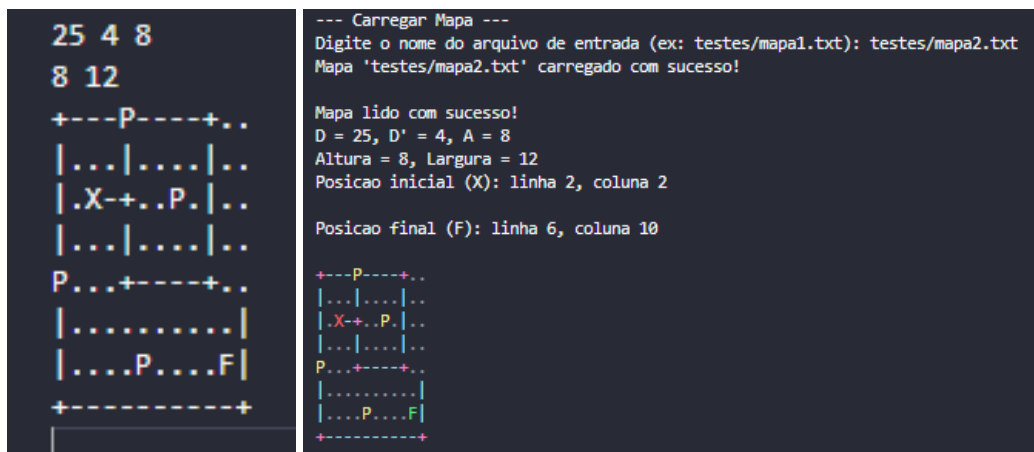
Para auxiliar o processo de compilação e execução do programa, foi criado um arquivo **Makefile**. Logo, uma vez que o terminal de execução se encontra no diretório raiz, ou seja, *'TP1_PROJETO_E_ANALISE_DE_ALGORITMOS'*, basta utilizar o comando *make* para gerar o executável *leitura_mapa.exe* e, em seguida, o comando *make run* para a execução do código compilado.

6. Resultados

Uma vez que o programa foi devidamente executado, é possível testar o seu funcionamento. Logo, nesta seção serão apresentados os resultados obtidos a partir das funcionalidades implementadas.

6.1 Carregamento do mapa

A figura 16 mostra que o programa conseguiu obter todas as informações necessárias a partir do arquivo de teste, criando o mapa com sucesso.



```

25 4 8
8 12
+---P---+..
|...|...|..
|.X-+..P.|..
|...|...|..
P...+---+..
|.....|
|...P...F|
+-----+

--- Carregar Mapa ---
Digite o nome do arquivo de entrada (ex: testes/mapa1.txt): testes/mapa2.txt
Mapa 'testes/mapa2.txt' carregado com sucesso!

Mapa lido com sucesso!
D = 25, D' = 4, A = 8
Altura = 8, Largura = 12
Posicao inicial (X): linha 2, coluna 2

Posicao final (F): linha 6, coluna 10

+---P---+..
|...|...|..
|.X-+..P.|..
|...|...|..
P...+---+..
|.....|
|...P...F|
+-----+

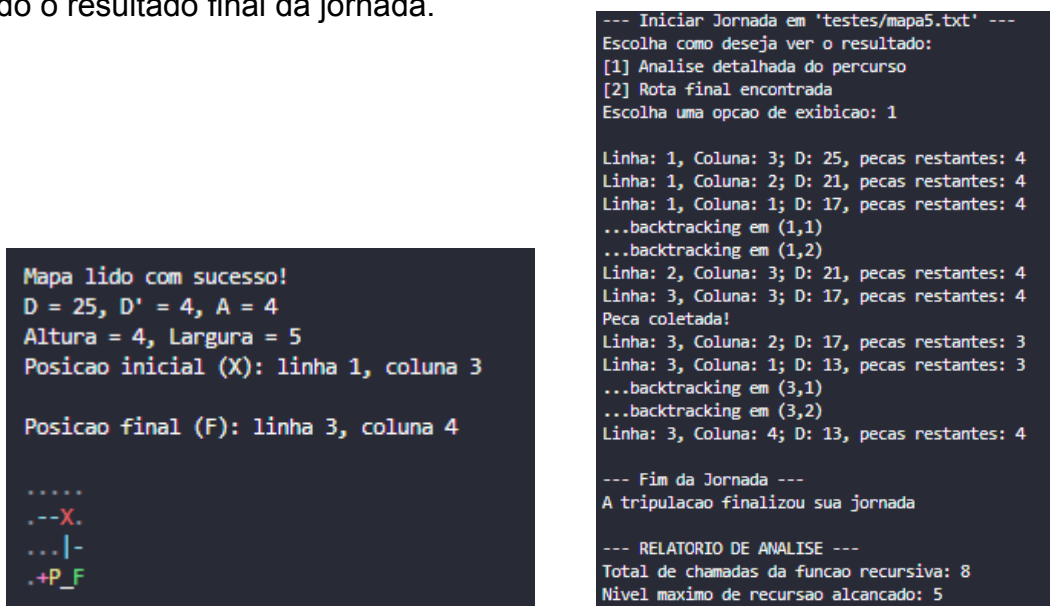
```

Figura 16 - Leitura e identificação do mapa

6.2 Jornada espacial

Agora serão detalhados diferentes resultados obtidos pela tripulação, de acordo com o percurso.

Existem duas maneiras de visualização desses resultados, a primeira opção, retratada na figura 17, representa uma análise mais aprofundada que mostra o percurso detalhadamente, ou seja, é possível visualizar inclusive as rotas alternativas e os pontos em que foi realizado o backtracking pela função e, ao fim, é exibido o resultado final da jornada.



```

Mapa lido com sucesso!
D = 25, D' = 4, A = 4
Altura = 4, Largura = 5
Posicao inicial (X): linha 1, coluna 3

Posicao final (F): linha 3, coluna 4

.....
.--X.
...|-
.+P_F

--- Iniciar Jornada em 'testes/mapa5.txt' ---
Escolha como deseja ver o resultado:
[1] Analise detalhada do percurso
[2] Rota final encontrada
Escolha uma opcao de exibicao: 1

Linha: 1, Coluna: 3; D: 25, pecas restantes: 4
Linha: 1, Coluna: 2; D: 21, pecas restantes: 4
Linha: 1, Coluna: 1; D: 17, pecas restantes: 4
...backtracking em (1,1)
...backtracking em (1,2)
Linha: 2, Coluna: 3; D: 21, pecas restantes: 4
Linha: 3, Coluna: 3; D: 17, pecas restantes: 4
Peca coletada!
Linha: 3, Coluna: 2; D: 17, pecas restantes: 3
Linha: 3, Coluna: 1; D: 13, pecas restantes: 3
...backtracking em (3,1)
...backtracking em (3,2)
Linha: 3, Coluna: 4; D: 13, pecas restantes: 4

--- Fim da Jornada ---
A tripulacao finalizou sua jornada

--- RELATORIO DE ANALISE ---
Total de chamadas da funcao recursiva: 8
Nivel maximo de recursao alcançado: 5

```

Figura 17 - Percurso detalhado de um caso de sucesso.

A segunda forma de visualizar o resultado deve ser utilizada para exibir apenas a rota possível, caso ela tenha sido encontrada. Portanto, como pode-se observar na figura 18, é exibido uma rota funcional que leva a nave diretamente ao seu destino, além disso existe uma representação visual do percurso, destacado no mapa pela cor vermelha, seguido do resultado da jornada.

```

Mapa lido com sucesso!
D = 20, D' = 5, A = 10
Altura = 7, Largura = 10
Posicao inicial (X): linha 1, coluna 2

Posicao final (F): linha 4, coluna 6

P-+...+--+
|.X...|.
+-+P-+..P
|.|...P..|
P.|...FP-+
|.|.....|
+-P-----+

--- Iniciar Jornada em 'testes/mapa1.txt' ---
Escolha como deseja ver o resultado:
[1] Analise detalhada do percurso
[2] Rota final encontrada
Escolha uma opcao de exibicao: 2

Linha: 1, Coluna: 2; D: 20, pecas restantes: 4
Linha: 2, Coluna: 2; D: 15, pecas restantes: 4
Linha: 2, Coluna: 3; D: 10, pecas restantes: 4
Linha: 2, Coluna: 4; D: 15, pecas restantes: 3
Linha: 2, Coluna: 5; D: 10, pecas restantes: 3
Linha: 2, Coluna: 6; D: 5, pecas restantes: 3
Linha: 3, Coluna: 6; D: 10, pecas restantes: 2
Linha: 4, Coluna: 6; D: 5, pecas restantes: 2

Rota:
P - + . . . + - - +
| . X . . . | . . |
+ - + - P - + . . P
| . | . . . P . . |
P . | . . . F P - +
| . | . . . . . |
+ - P - - - - - +

--- Fim da Jornada ---
A tripulacao finalizou sua jornada

--- RELATORIO DE ANALISE ---
Total de chamadas da funcao recursiva: 20
Nivel maximo de recursao alcancado: 8

```

Figura 18 - Exibição da rota encontrada.

A seguir, pela figura 19, está retratada uma situação em que a nave não consegue encontrar um caminho que alcance o seu destino, resultando em uma falha.

```
Mapa lido com sucesso!
D = 20, D' = 2, A = 1
Altura = 7, Largura = 10
Posicao inicial (X): linha 0, coluna 1

Posicao final (F): linha 4, coluna 7

|X|---|+.-
#-|||---+
.-...|---
.-...|---
--P.|P.F|-
P|-..P+-P|
---P....-
```

```
Escolha uma opcao de exibicao: 1

Linha: 0, Coluna: 1; D: 20, pecas restantes: 4
Linha: 0, Coluna: 0; D: 18, pecas restantes: 4
Em meio a chuva de meteoros, um acertou a nave!
Linha: 1, Coluna: 0; D: 14, pecas restantes: 4
Linha: 1, Coluna: 1; D: 12, pecas restantes: 4
Linha: 1, Coluna: 2; D: 10, pecas restantes: 4
Linha: 0, Coluna: 2; D: 8, pecas restantes: 4
...backtracking em (0,2)
...backtracking em (1,2)
...backtracking em (1,1)
...backtracking em (1,0)
...backtracking em (0,0)
Linha: 0, Coluna: 2; D: 18, pecas restantes: 4
Linha: 1, Coluna: 2; D: 16, pecas restantes: 4
...backtracking em (1,2)
...backtracking em (0,2)
Linha: 1, Coluna: 1; D: 18, pecas restantes: 4
Em meio a chuva de meteoros, um acertou a nave!
Linha: 1, Coluna: 0; D: 14, pecas restantes: 4
Linha: 0, Coluna: 0; D: 12, pecas restantes: 4
...backtracking em (0,0)
...backtracking em (1,0)
Linha: 1, Coluna: 2; D: 16, pecas restantes: 4
Linha: 0, Coluna: 2; D: 14, pecas restantes: 4
...backtracking em (0,2)
...backtracking em (1,2)
...backtracking em (1,1)
...backtracking em (0,1)

--- Fim da Jornada ---
Apesar da bravura a tripulacao falhou em sua jornada

--- RELATORIO DE ANALISE ---
Total de chamadas da funcao recursiva: 13
Nivel maximo de recursao alcançado: 6
```

Figura 19 - Representação de uma jornada que resultou em falha.

E, por fim, é possível acontecer uma situação em que a nave consegue coletar quatro peças e ser consertada, garantindo o sucesso na jornada caso exista um caminho que conecte até o destino final (pois pode existir um mapa em que quatro peças são coletadas porém aquela rota leva a um beco sem saída, impossibilitando a tripulação de alcançar o destino apesar da nave em bom estado) como mostra a figura 20.

```

Mapa lido com sucesso!
D = 25, D' = 4, A = 8
Altura = 8, Largura = 12
Posicao inicial (X): linha 2, coluna 2

Posicao final (F): linha 6, coluna 10

P---P----+..
|...|....|..
P.X-+.P.|..
|...|....|..
P...+----+..
|.....|
|...P...F|
+-----|

Linha: 0, Coluna: 2; D: 9, pecas restantes: 3
Linha: 0, Coluna: 1; D: 5, pecas restantes: 3
Linha: 0, Coluna: 0; D: 9, pecas restantes: 2
Linha: 1, Coluna: 0; D: 5, pecas restantes: 2
Linha: 2, Coluna: 0; D: 9, pecas restantes: 1
Linha: 3, Coluna: 0; D: 5, pecas restantes: 1
Linha: 4, Coluna: 0; D: 9, pecas restantes: 0
Linha: 5, Coluna: 0; D: 9, pecas restantes: 0
Linha: 6, Coluna: 0; D: 9, pecas restantes: 0
Linha: 7, Coluna: 0; D: 9, pecas restantes: 0
Linha: 7, Coluna: 1; D: 9, pecas restantes: 0
Linha: 7, Coluna: 2; D: 9, pecas restantes: 0
Linha: 7, Coluna: 3; D: 9, pecas restantes: 0
Linha: 7, Coluna: 4; D: 9, pecas restantes: 0
Linha: 7, Coluna: 5; D: 9, pecas restantes: 0
Linha: 7, Coluna: 6; D: 9, pecas restantes: 0
Linha: 7, Coluna: 7; D: 9, pecas restantes: 0
Linha: 7, Coluna: 8; D: 9, pecas restantes: 0
Linha: 7, Coluna: 9; D: 9, pecas restantes: 0
Linha: 7, Coluna: 10; D: 9, pecas restantes: 0
Linha: 6, Coluna: 10; D: 9, pecas restantes: 0

Rota:
P - - - P - - - + . .
| . . . | . . . | . .
P . X - + . . P . | . .
| . . . | . . . | . .
P . . . + - - - + . .
| . . . . . . . . |
| . . . . P . . . . F |
+ - - - - - - - - | .

--- Fim da Jornada ---
A jornada sera finalizada sem mais desafios

--- RELATORIO DE ANALISE ---
Total de chamadas da funcao recursiva: 27
Nivel maximo de recursao alcancado: 27

```

Figura 20 - Situação em que a nave é consertada durante o percurso.

6.3 Geração de mapas

Outra funcionalidade do programa é a capacidade de criação de mapas, que a partir das informações solicitadas ao usuário, gera uma matriz que pode ser percorrida pela nave. A representação de seu funcionamento está exibida na figura 21.

```
=== GERADOR DE MAPAS AUTOMATICOS ===

Durabilidade inicial (D): 20
Perda por movimento (D'): 5
Ganho por peca (A): 10
Altura do mapa: 10
Largura do mapa: 10
Quantidade de pecas (P): 6
Nível de dificuldade (1 = facil, 2 = medio, 3 = dificil): 1
Nome do arquivo de saída (ex: testes/mapaN.txt, sendo N um numero): testes/mapa6.txt

Arquivo 'testes/mapa6.txt' gerado com sucesso!

Mapa lido com sucesso!
D = 20, D' = 5, A = 10
Altura = 10, Largura = 10
Posicao inicial (X): linha 1, coluna 5

Posicao final (F): linha 0, coluna 3

|-|F|--+-.
||+P-X--.|
|.---.---|.
-|--P.----
-|-|-|----
+++|---P|
|-+|-.-.---
P+-----+P--
+|--P-----#
||-----+-

Mapa gerado! Pressione Enter para voltar ao menu principal...[]
```

Figura 21 - Geração de um novo mapa.

6.4 Obstáculos

A seguir, na figura 22, é possível observar a ocorrência de eventos, como a chuva de meteoros e o roubo alienígena que dificultam o percurso da tripulação.


```
Mapa lido com sucesso!
D = 30, D' = 5, A = 10
Altura = 6, Largura = 8
Posicao inicial (X): linha 1, coluna 1

Posicao final (F): linha 4, coluna 6

.....
.X-|-P..
.P...#..
.#.-+&.P
.&-+--F.
.....

Escolha uma opcao de exibicao: 1

Linha: 1, Coluna: 1; D: 30, pecas restantes: 4
Linha: 1, Coluna: 2; D: 25, pecas restantes: 4
Linha: 1, Coluna: 3; D: 20, pecas restantes: 4
...backtracking em (1,3)
...backtracking em (1,2)
Peca coletada!
Linha: 2, Coluna: 1; D: 35, pecas restantes: 3
Em meio a chuva de meteoros, um acertou a nave!
Linha: 3, Coluna: 1; D: 25, pecas restantes: 3
Os alienigenas roubaram = 1 peca(s)!
Linha: 4, Coluna: 1; D: 20, pecas restantes: 4
Linha: 4, Coluna: 2; D: 15, pecas restantes: 4
Linha: 4, Coluna: 3; D: 10, pecas restantes: 4
Linha: 4, Coluna: 4; D: 5, pecas restantes: 4
Linha: 4, Coluna: 5; D: 0, pecas restantes: 4
...backtracking em (4,5)
...backtracking em (4,4)
Linha: 3, Coluna: 3; D: 5, pecas restantes: 4
Linha: 3, Coluna: 4; D: 0, pecas restantes: 4
...backtracking em (3,4)
...backtracking em (3,3)
...backtracking em (4,3)
...backtracking em (4,2)
...backtracking em (4,1)
...backtracking em (3,1)
...backtracking em (2,1)
...backtracking em (1,1)

--- Fim da Jornada ---
Apesar da bravura a tripulacao falhou em sua jornada
```

Figura 22 - Obstáculos no percurso.

7. Conclusão

Dessa forma, observa-se que a implementação do algoritmo de Backtracking, aliada às estruturas dinâmicas de dados e às funcionalidades adicionais desenvolvidas, mostrou-se eficiente e plenamente adequada às especificações do trabalho proposto. A abordagem recursiva adotada foi fundamental para a exploração completa das possíveis rotas até o planeta das festividades, permitindo compreender de forma prática o funcionamento do paradigma de busca e retrocesso.

Diante disso, é possível concluir que a implementação desse projeto configurou-se como uma experiência muito enriquecedora, uma vez que possibilitou a aplicação de conceitos e de um algoritmo visto em sala de aula, contribuindo assim, para uma aprendizagem mais profunda a respeito da disciplina de Projetos e Análise de Algoritmos.

8. Referências

[1] Github. Disponível em: <<https://github.com/>>.

[2] Solving a labyrinth with backtracking. Disponível em: <<https://www.linkedin.com/pulse/solving-labyrinth-backtracking-guide-enhancements-allan-cruz-tlodf>> Acesso em 15 out. 2025.

[3] Notas de aula.